

# Module 4

## Basic Testbench Construction

# Learning objectives

- Master construction of structured testbenches with proper organization

# Prerequisites

- See module README

# Learning path

## Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["Testbench Organization"]

T2["Clock and Reset Generation"]

T1 --> T2

T3["Stimulus Generation"]

T2 --> T3

Master construction of structured testbenches with proper organization

# Overview

- This module focuses on building well-structured testbenches for both Verilog and C++ paradigms. You...

# Design architecture

# RTL / block architecture

## Rtl Architecture

Diagram: Rtl Architecture

(render with mmdc when available)

flowchart LR

subgraph agents["Modular TB"]

CLK[clock\_reset]

STIM[stimulus]

CHK[checker]

end

Module 4: DUT / block hierarchy (see module4/examples/).

# Verification environment architecture

## Verification Architecture

Diagram: Verification Architecture

(render with mmdc when available)

flowchart LR

SCEN[scenarios] --> AGENTS[agents]

AGENTS --> REF[reference model]

REF --> SELF[self-check]

Module 4: UVM layers, agents, and checkers for this phase.



# 1. Structured DUT portfolio (Module 4)

- ``dut/alus/simple_alu.v``: Combinational ALU — op select, result, zero flag
- ``dut/fifos/simple_fifo.v``: Queue storage with full/empty and pointer logic
- ``dut/registers/register_file.v``: Multi-port register file — read/write ports, clocked updates
- Complexity step: Moves from gates/counters to datapath blocks needing organized TBs

## 2. Modular testbench architecture

- Separation: Stimulus generator, monitor, checker/scoreboard as distinct modules or classes
- Clock/reset generator: Dedicated block (``clock_reset`` examples) — configurable period
- Hierarchy: Top TB instantiates agents + DUT; connects via named interfaces (wires)
- C++ mirror: Same roles as C++ classes with methods ``drive()``, ``sample()``, ``check()``

### 3. Reference model placement

- Golden model: Software replica of ALU/FIFO/register behavior in TB
- Self-checking path: DUT output compared to reference each cycle or transaction
- Config: Parameters for width, depth, and clock period without editing DUT RTL

# **Verification & testing methods**

# Testing and closure flow

## Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart TD

A[modular agents] --> B[scenarios]

B --> C[self-check report]

Module 4: how tests, checks, and metrics close verification goals.

# 1. Component-level verification

- Stimulus module: Encapsulates write sequences to FIFO or register ports
- Monitor module: Captures read data and status flags passively
- Checker: Flags protocol violations (overflow, X on outputs, wrong opcode result)

## 2. Scenario-based testing

- Scenarios: Reset + fill FIFO; ALU op sweep; register R/W hazard patterns
- Tasks/methods: Each scenario is a callable task — reusable across tests
- Aggregation: Single report summarizing scenarios run and failures

### 3. Structured debug and closure

- Modular debug: Isolate failing agent by disabling others in top TB
- Waveform on demand: Trigger dumps only when checker fires
- Regression: ``./scripts/module4.sh --self-checking`` and ``--all-tests`` for batch sign-off



# Syllabus topics

# 1. Testbench Organization (1/7)

- Modular Testbench Structure (Verilog and C++)
- Separating concerns
- Component-based design
- Reusability principles
- Maintainability
- Separate Stimulus, Monitor, and Checker Components

# 1. Testbench Organization (2/7)

- Stimulus generation module/class
- Response monitoring module/class
- Result checking module/class
- Component interaction
- Verilog: Module-Based Organization
- Module hierarchy

# 1. Testbench Organization (3/7)

- Module instantiation
- Signal connectivity
- Module reusability
- C++: Class-Based Organization
- Class hierarchy
- Class instantiation

# 1. Testbench Organization (4/7)

- Method organization
- Class reusability
- Testbench Component Organization
- Component placement
- Signal routing
- Hierarchy design

# 1. Testbench Organization (5/7)

- Organization patterns
- Reusable Testbench Components
- Component design
- Parameterization
- Configuration
- Reuse strategies

# 1. Testbench Organization (6/7)

- Configuration and Parameters
- Parameter passing
- Configuration management
- Runtime configuration
- Build-time configuration
- ``register_file_test_verilog.v``: Structured Verilog testbench with separate modules

# 1. Testbench Organization (7/7)

- ``register_file_test_cpp.cpp``: Structured C++ testbench with separate classes



## 2. Clock and Reset Generation (1/8)

- Verilog: Clock Generation Patterns (Always Blocks)
- Continuous clock generation
- Configurable periods
- Multiple clocks
- Clock gating
- C++: Clock Generation Patterns (Simulation Loop)

## 2. Clock and Reset Generation (2/8)

- Clock toggling in loop
- Configurable periods
- Multiple clocks
- Clock coordination
- Configurable Clock Periods (Both Paradigms)
- Parameter-based configuration

## 2. Clock and Reset Generation (3/8)

- Runtime configuration
- Clock period calculation
- Frequency control
- Multiple Clock Domains
- Independent clocks
- Clock relationships

## 2. Clock and Reset Generation (4/8)

- Domain crossing
- Synchronization
- Reset Generation (Synchronous, Asynchronous)
- Synchronous reset
- Asynchronous reset
- Reset timing

## 2. Clock and Reset Generation (5/8)

- Reset sequences
- Verilog: Reset in Initial Blocks
- Initial block reset
- Reset timing control
- Reset sequences
- Reset coordination

## 2. Clock and Reset Generation (6/8)

- C++: Reset in C++ Code
- Reset in simulation loop
- Reset timing control
- Reset sequences
- Reset coordination
- Reset Sequences and Timing

## 2. Clock and Reset Generation (7/8)

- Reset assertion
- Reset deassertion
- Reset duration
- Reset timing
- Clock and Reset Coordination
- Clock/reset relationship

## 2. Clock and Reset Generation (8/8)

- Reset synchronization
- Timing coordination
- Best practices
- ``configurable_clock_reset_verilog.v``: Configurable clock and reset in Verilog
- ``configurable_clock_reset_cpp.cpp``: Configurable clock and reset in C++



### 3. Stimulus Generation (1/5)

- Test Vector Generation
- Vector format
- Vector generation
- Vector storage
- Vector application
- Pattern Generation

### 3. Stimulus Generation (2/5)

- Pattern types
- Pattern generation
- Pattern application
- Pattern verification
- Sequential Stimulus Application
- Sequential application

### 3. Stimulus Generation (3/5)

- Timing control
- Application order
- Synchronization
- Stimulus Timing Control
- Timing accuracy
- Timing relationships

### 3. Stimulus Generation (4/5)

- Timing control
- Timing verification
- Stimulus Verification
- Stimulus validation
- Stimulus checking
- Error detection

### 3. Stimulus Generation (5/5)

- Verification strategies

## 4. Response Monitoring (1/5)

- Output Monitoring Strategies
- Real-time monitoring
- Post-processing monitoring
- Event-driven monitoring
- Continuous monitoring
- Real-Time Monitoring

## 4. Response Monitoring (2/5)

- Immediate monitoring
- Live updates
- Real-time analysis
- Performance considerations
- Post-Processing Monitoring
- Delayed analysis

## 4. Response Monitoring (3/5)

- Batch processing
- Data collection
- Analysis strategies
- Monitoring Timing
- When to monitor
- Timing accuracy



## 4. Response Monitoring (4/5)

- Timing relationships
- Timing control
- Event-Driven Monitoring
- Event triggers
- Event-based monitoring
- Event handling

## 4. Response Monitoring (5/5)

- Event coordination

## 5. Result Checking (1/5)

- Expected Value Calculation
- Expected value computation
- Reference models
- Calculation methods
- Accuracy
- Output Comparison

## 5. Result Checking (2/5)

- Value comparison
- Comparison methods
- Comparison timing
- Comparison accuracy
- Error Detection and Reporting
- Error detection

## 5. Result Checking (3/5)

- Error reporting
- Error handling
- Error analysis
- Assertion-Based Checking
- Assertion types
- Assertion implementation

## 5. Result Checking (4/5)

- Assertion organization
- Assertion best practices
- Self-Checking Testbenches
- Automatic checking
- Self-verification
- Result aggregation

## 5. Result Checking (5/5)

- Pass/fail determination
- ``alu_self_checking_verilog.v``: Self-checking ALU testbench in Verilog
- ``alu_self_checking_cpp.cpp``: Self-checking ALU testbench in C++

## 6. Test Scenarios (1/7)

- Test Case Organization
- Test case structure
- Test case organization
- Test case management
- Test case documentation
- Multiple Test Scenarios



## 6. Test Scenarios (2/7)

- Scenario definition
- Scenario organization
- Scenario selection
- Scenario execution
- Test Sequencing
- Test order

## 6. Test Scenarios (3/7)

- Test dependencies
- Test sequencing
- Test coordination
- Test Selection Mechanisms
- Test selection
- Test filtering

## 6. Test Scenarios (4/7)

- Test prioritization
- Test management
- Test Result Aggregation
- Result collection
- Result aggregation
- Result analysis

## 6. Test Scenarios (5/7)

- Result reporting
- Location: ``module4/examples/modular_testbenches/register_file_test_verilog.v``
- Demonstrates: Modular structure, separate stimulus/monitor/checker modules
- Location: ``module4/examples/modular_testbenches/register_file_test_cpp.cpp``
- Demonstrates: Class-based organization, separate stimulus/monitor/checker classes
- Location: (Coming soon)

## 6. Test Scenarios (6/7)

- Demonstrates: Multiple test scenarios, test sequencing
- Location: (Coming soon)
- Demonstrates: Multiple test scenarios, test sequencing
- Location: ``module4/examples/self_checking/``
- Demonstrates: Self-checking testbenches, comprehensive testing
- Location: (Coming soon)

## 6. Test Scenarios (7/7)

- Demonstrates: State machine testing, state coverage
- Location: ``module4/examples/clock_reset/``
- Demonstrates: Configurable clock/reset, parameterization

# Hands-on examples

# Module 4 self-check

```
$ ./scripts/module4.sh --check
```

Terminal: module4\_check

```
=====
Module 4: environment self-check
=====
[OK] module4/ present
[OK] module4/examples/ present
[OK] iverilog: Icarus Verilog version 12.0 (stable) ()
[INFO] verilator not installed (needed for Module 2+ labs: ./scripts/install_verilator.sh)
All required checks passed
```



# Exercise scaffold

```
./scripts/module4.sh --scaffold
```

# Demo: Clock and reset

```
$ cd module4/examples/clock_reset && make all
```

Terminal: ex01\_clock\_reset

```
=== Compiling Verilog clock/reset example ===
iverilog -o configurable_clock_reset configurable_clock_reset_verilog.v
=== Running simulation ===
vvp configurable_clock_reset
=====
Configurable Clock and Reset Example
=====
Clock 1 period: 20 ns (50.0 MHz)
Clock 2 period: 40 ns (25.0 MHz)
Reset duration: 100 ns
=====
Time 0: Reset asserted
VCD info: dumpfile configurable_clock_reset.vcd opened for output.
Time 10000: Clock 1 posedge
Time 20000: Clock 2 posedge
Time 30000: Clock 1 posedge
Time 50000: Clock 1 posedge
Time 60000: Clock 2 posedge
Time 70000: Clock 1 posedge
Time 90000: Clock 1 posedge
Time 100000: Reset released
Time 100000: Clock 2 posedge
Time 110000: Clock 1 posedge
Time 130000: Clock 1 posedge
Time 140000: Clock 2 posedge
Time 150000: Clock 1 posedge
Time 170000: Clock 1 posedge
Time 180000: Clock 2 posedge
Time 190000: Clock 1 posedge
Time 210000: Clock 1 posedge
Time 220000: Clock 2 posedge
Time 230000: Clock 1 posedge
Time 250000: Clock 1 posedge
Time 260000: Clock 2 posedge
Time 270000: Clock 1 posedge
Time 290000: Clock 1 posedge
```

# Demo: Self-checking

```
$ cd module4/examples/self_checking && make all
```

Terminal: ex02\_self\_checking

```
=== Compiling Verilog self-checking testbench ===
iverilog -o alu_self_checking alu_self_checking_verilog.v ../module4/dut/alus/simple_alu.v
=== Running simulation ===
vvp alu_self_checking
=====
Self-Checking ALU Testbench
=====
VCD info: dumpfile alu_self_checking.vcd opened for output.
[PASS] Test 1: ADD a=0x10, b=0x20, result=0x30, zero=0
[PASS] Test 2: ADD a=0xff, b=0x01, result=0x00, zero=1
[PASS] Test 3: ADD a=0x00, b=0x00, result=0x00, zero=1
[PASS] Test 4: SUB a=0x30, b=0x10, result=0x20, zero=0
[PASS] Test 5: SUB a=0x00, b=0x01, result=0xff, zero=0
[PASS] Test 6: AND a=0xaa, b=0x55, result=0x00, zero=1
[PASS] Test 7: AND a=0xff, b=0xff, result=0xff, zero=0
[PASS] Test 8: AND a=0x00, b=0xff, result=0x00, zero=1
[PASS] Test 9: OR a=0xaa, b=0x55, result=0xff, zero=0
[PASS] Test 10: OR a=0x00, b=0x00, result=0x00, zero=1

=====
Test Summary
=====
Total tests: 10
Passed:      10
Failed:      0
=====
✓ All tests PASSED!
=====
alu_self_checking_verilog.v:288: $finish called at 60000 (1ps)
=== Compiling C++ self-checking testbench ===
verilator --cc --exe --build -I../module4/dut/alus \
[]../module4/dut/alus/simple_alu.v alu_self_checking_cpp.cpp
make: verilator: No such file or directory
make: *** [Makefile:18: alu_self_checking_cpp] Error 127
```

# Demo: Stimulus and monitoring

```
$ cd module4/examples/stimulus_monitoring && make all
```

Terminal: ex03\_stimulus\_monitoring

```
=====
Stimulus/Monitoring Examples
=====
Stimulus/monitoring examples are coming soon.
See module4/README.md for details.
=====
```

# Demo: Modular testbenches

```
$ cd module4/examples/modular_testbenches && make all
```

Terminal: ex04\_modular\_testbenches

```
=== Compiling Verilog modular testbench ===
iverilog -o register_file_test_verilog \
  register_file_test_verilog.v \
  .././../module4/dut/registers/register_file.v
=== Running simulation ===
vvp register_file_test_verilog
VCD info: dumpfile register_file_test.vcd opened for output.
[MONITOR] Time 10000: Read addr0=0 -> data0=0xxx, addr1=0 -> data1=0xxx
[MONITOR] Time 30000: Read addr0=0 -> data0=0x00, addr1=0 -> data1=0x00
ERROR: register_file_test_verilog.v:350: [CHECKER] Time 30000: Read0 mismatch! Addr=0, Expected=
Time: 30000 Scope: register_file_test_verilog.chk
ERROR: register_file_test_verilog.v:357: [CHECKER] Time 30000: Read1 mismatch! Addr=0, Expected=
Time: 30000 Scope: register_file_test_verilog.chk
ERROR: register_file_test_verilog.v:350: [CHECKER] Time 50000: Read0 mismatch! Addr=0, Expected=
Time: 50000 Scope: register_file_test_verilog.chk
ERROR: register_file_test_verilog.v:357: [CHECKER] Time 50000: Read1 mismatch! Addr=0, Expected=
Time: 50000 Scope: register_file_test_verilog.chk
[MONITOR] Time 50000: Write addr=0, data=0xaa
[MONITOR] Time 50000: Read addr0=0 -> data0=0x00, addr1=0 -> data1=0x00
[MONITOR] Time 70000: Write addr=1, data=0xbb
[MONITOR] Time 70000: Read addr0=0 -> data0=0xaa, addr1=0 -> data1=0xaa
[MONITOR] Time 90000: Write addr=2, data=0xcc
[MONITOR] Time 90000: Read addr0=0 -> data0=0xaa, addr1=0 -> data1=0xaa
[MONITOR] Time 110000: Read addr0=0 -> data0=0xaa, addr1=1 -> data1=0xbb
ERROR: register_file_test_verilog.v:357: [CHECKER] Time 130000: Read1 mismatch! Addr=3, Expected=
Time: 130000 Scope: register_file_test_verilog.chk
[MONITOR] Time 130000: Read addr0=2 -> data0=0xcc, addr1=3 -> data1=0x00
[MONITOR] Time 150000: Read addr0=2 -> data0=0xcc, addr1=3 -> data1=0x00
ERROR: register_file_test_verilog.v:357: [CHECKER] Time 150000: Read1 mismatch! Addr=3, Expected=
Time: 150000 Scope: register_file_test_verilog.chk
ERROR: register_file_test_verilog.v:357: [CHECKER] Time 170000: Read1 mismatch! Addr=3, Expected=
Time: 170000 Scope: register_file_test_verilog.chk
[MONITOR] Time 170000: Read addr0=2 -> data0=0xcc, addr1=3 -> data1=0x00
[MONITOR] Time 190000: Read addr0=2 -> data0=0xcc, addr1=3 -> data1=0x00
ERROR: register_file_test_verilog.v:357: [CHECKER] Time 190000: Read1 mismatch! Addr=3, Expected=
Time: 190000 Scope: register_file_test_verilog.chk
```

# Practice & assessment

# Assessment checklist

- Can organize testbenches modularly (both paradigms)
- Can generate configurable clocks and resets (Verilog and C++)
- Can create structured stimulus (both paradigms)
- Can implement monitoring strategies (both paradigms)
- Can build self-checking testbenches (both paradigms)
- Can organize multiple test scenarios

# Summary & next steps

- Master construction of structured testbenches with proper organization
- Complete module4/CHECKLIST.md
- Run `./scripts/module4.sh --check`
- Next: Next module in course